

Assignment 4: BBS Frontend

Software Engineering - UATX - Spring 2026

Due	Friday, May 15, 2026, 5:00pm
Prereqs	Assignment 2 (BBS API) and lectures 5.1 (JS & the browser), 5.2 (talking to your server), 6.1 (React intro), 7.1 (state and effects)
Submit	Branch + PR on the class repo

Background

Your BBS API from A2 is a real web server, but right now the only thing talking to it is curl and `verify_api.py`. In this assignment you build the actual product on top: a React app in the browser that lets a person log in by username, read the feed, post messages, click into other users' profiles, and so on. The same backend you already have, with a real frontend in front of it.

A2 had `verify_api.py` to tell you whether you got it right...this one doesn't, because browsers are visual, stateful, asynchronous, and most of what makes a frontend "good" is stuff a script can't really check. So a bigger share of the grade rides on decisions you make and on whether your README convinces me you understood the thing your agent built. The bronze bar is higher than A2's was, and the silver tier asks for real architectural choices (routing, state/cache, identity persistence) rather than just bolt-on features.

Setup

You'll work in a new directory next to your A2, in the same class repo. Your A2 stays put at `assignments/bbs-webserver/<your-github-username>/` and keeps running as the backend; your A4 frontend lives alongside it at `assignments/bbs-frontend/<your-github-username>/`. Inside the A4 directory:

```
src/           # your React code
public/
tests/         # silver+ - vitest and/or playwright
package.json
vite.config.ts
tsconfig.json
index.html
README.md
```

We will use **React + TypeScript + Vite**, which is what lectures 6.1 and 7.1 used. From the repo root:

```
mkdir -p assignments/bbs-frontend/<your-github-username>
cd assignments/bbs-frontend/<your-github-username>
npm create vite@latest . -- --template react-ts
npm install
npm run dev
```

The `.` in the create-vite command tells it to scaffold *into* the current directory rather than make a new `bbs-frontend/` subfolder underneath it.

Running the backend and frontend together

You will have **two processes** going while you develop - one for your A2 backend, one for your A4 frontend. Open two terminal tabs.

Terminal 1, your A2 backend (same command and directory you used in A2):

```
cd assignments/bbs-webserver/<your-github-username>
# activate your A2 venv however you set it up
uvicorn main:app --port 8000
```

Terminal 2, your A4 frontend:

```
cd assignments/bbs-frontend/<your-github-username>
npm run dev
```

Vite will print a URL like `http://localhost:5173` ; that's what you open in the browser. The frontend reads the backend URL from the `VITE_API_BASE` env var, defaulting to `http://localhost:8000` .

Which API do I point at?

Use your own A2 API on `http://localhost:8000`. The base URL should come from a `VITE_API_BASE` env var (defaulting to `http://localhost:8000`), so you can swap it without code changes when you deploy. Yes, your A2 - the whole point of putting a frontend on top of your own backend is that you own the contract end to end, and you'll find things about your A2 by actually using it that you never noticed when only `verify_api.py` was talking to it. If your A2 has bugs that this exercise surfaces, fix them there and note the fix in your README.

CORS. We did not cover CORS in class, but you are about to meet it. The very short version: when a page at `localhost:5173` tries to `fetch` from `localhost:8000`, the browser sees those as different "origins" and, by default, refuses to let the JavaScript on the page see the response unless the server explicitly opts in. This is a security feature, not a bug. The browser is protecting your users from a malicious site stealing data from a logged-in tab on another site. The [FastAPI CORS docs](#) are the better primer here - they explain what CORS is and give you the exact `CORSMiddleware` snippet to drop into your A2 `main.py` in one page. Mention this in your README under "changes I made to my A2 backend".

What the app does

Pages and what they show

You need at least these views. They can be routes (silver) or just conditional renders (bronze).

View	What it shows
Feed	The N most recent posts from <code>GET /posts</code> , newest first. Each row shows the message, the username (clickable to that user's profile), and the timestamp. A search box that hits <code>?q=</code> . Pagination controls (next/prev or "load more").
Compose	A textarea + post button that calls <code>POST /posts</code> with the current user's <code>X-Username</code> header. Shows server-side validation errors (422 with message length, etc) inline next to the field. Cmd+Enter / Ctrl+Enter posts.
User list	<code>GET /users</code> rendered as a list of clickable usernames.

User profile	Single user's info from <code>GET /users/{username}</code> plus their posts from <code>GET /users/{username}/posts</code> . 404 view if the user doesn't exist.
Post detail	<code>GET /posts/{id}</code> . Shows the post and a delete button (visible to anyone in bronze - this is fine, <code>X-Username</code> is not real auth, see your A2 README; silver asks you to be identity-aware and only show it on your own posts).
Sign-up / switch user	A way to create a new user (<code>POST /users</code>) and a way to switch which username you're "logged in as". Persisted in <code>localStorage</code> so a refresh doesn't kick you out.

localStorage. Same deal as CORS - we did not cover localStorage in lecture either, but you'll want it for the "stay logged in across refreshes" piece. It's a key/value store the browser keeps per-origin (so your app on `localhost:5173` has its own bucket, separate from anything else you've ever opened). The whole API you need is `localStorage.setItem('username', 'alice')` and `localStorage.getItem('username')` - both synchronous, both string-or-null. It survives refreshes and tab closes, which is what makes the "remember which user I am" requirement basically a one-liner. It does NOT survive a user clearing site data or browsing in incognito, which is fine - `X-Username` isn't real auth anyway, so we're just storing a preference.

Loading and error states

Every fetch in your app needs three states visible to the user:

1. **Loading** - some indication that a request is in flight. A spinner, "loading...", a skeleton row, anything that isn't blank.
2. **Success** - the data is rendered.
3. **Error** - if the request fails (network error, 4xx, 5xx), the user sees an actual message about what went wrong, not a blank screen and a console error. 422s from your A2 API have a useful `detail` field - surface it.

This is a thing agents are bad at by default. They will happily ship you a component that renders `data.map(...)` with no guard for the loading state and no error handling at all, and it will work fine in the happy path until you tab away and come back, at which point you

stare at a white page. Push your agent on this. Or write the loading/error scaffolding yourself first and ask the agent to fill in the success state.

Forms and validation

Your A2 API enforces rules: messages 1-500 chars, usernames 3-20 chars matching `^[a-zA-Z0-9_]+$`. Your frontend should help the user not hit those errors:

- Disable the submit button when the input is empty (or, for username, when it doesn't match the regex).
- Show a character count on the post textarea, turning red past 500.
- If the server still returns 422 (because someone's clever and bypassed the disabled state, or because the server has a stricter rule you didn't know about), show the server's error message inline. Do not eat it.

Accessibility - silver and gold

We didn't cover accessibility in lecture, so this isn't a bronze requirement. For silver and gold, do the basics: every input has a real `<label>` tied to it (with `htmlFor / id`, or by wrapping). Every button has visible text or an `aria-label`. The page should be usable from the keyboard - tab order makes sense, the post button is reachable, the search field is reachable. Nothing exhaustive...but if you can't tab to your own compose textarea, that's a problem.

Tiers

Bronze

All six views above are implemented and reachable. All eight A2 bronze endpoints are wired up somewhere in the UI. Every fetch has visible loading + error states. Compose form does client-side validation and surfaces server 422s. Current username persists across refresh via `localStorage`. README is present. The app runs with `npm install && npm run dev` against a default `VITE_API_BASE`.

Silver

Everything in bronze, plus all of the following:

- **Real routing.** Use `react-router-dom` (or equivalent). URLs like `/`, `/users`, `/users/alice`, `/posts/42` work, are bookmarkable, and survive a refresh. The browser back button does the right thing.

- **Optimistic updates** for at least one action - posting a new message OR deleting a post. The UI reflects the change immediately, then reconciles with the server response, and rolls back on failure (with an error toast or inline message).
- **Identity-aware destructive actions.** Don't put a delete button on a post the current user didn't write. This is a UX/clarity call, not security - `X-Username` still isn't real auth and your backend may still accept the call - but a good frontend doesn't dangle a destructive action in front of someone who has no business taking it. (Bronze lets you show delete to anyone; silver asks you to be identity-aware about it.)
- **Pagination UX** that feels right - infinite scroll, a "load more" button, numbered pages, your call. If your A2 only supports offset and you want cursor (or vice versa), feel free to change your A2; just mention the change in your README.
- **At least one keyboard shortcut** beyond `Cmd+Enter` to post - and it has to actually *do* something in the app: focus the search box, jump to the compose field, move through the feed, that kind of thing. Surface it somewhere visible (a "?" overlay, a tooltip, a footer line - not buried). A shortcut whose only job is to open the shortcuts panel does not count - that is the surfacing, not the shortcut.
- **A test directory** with at least 3 tests. Vitest + React Testing Library on a couple of components, OR a Playwright spec hitting `npm run dev`. Doesn't need 100% coverage, just real tests of real behavior.
- **Basic accessibility.** See the section above - labels, keyboard reachability, no `div`s pretending to be buttons.

Gold

Everything in silver, plus at least **two** of the following four. Pick the ones you actually want to dig into - you'll do better work on two you care about than four you're grinding through.

- **Real-time-ish updates.** When user A posts a message, user B's open feed reflects it within ~5 seconds without a manual refresh. Polling (`setInterval` + `refetch`) is fine and is the easiest path. Server-sent events or websockets are cooler but require backend work in your A2 server. Whichever you pick, explain in the README why polling-vs-push fits your use case.
- **One nontrivial UI feature you came up with.** Examples: dark mode that respects `prefers-color-scheme` and persists, drag-to-reorder draft messages, message reactions UI (if you built reactions in A2 gold), threads, mentions that render as links. Not a styling tweak - something with state and real interaction. Describe it in the README.

- **Real automated tests that prove the user flow.** A Playwright spec that runs against a local `npm run dev` (or a CI job that spins one up) and covers at least: create user, switch to that user, post a message, see it in the feed, delete it. Pasted into `tests/` and runnable with one command.
- **Visual design that has a point of view.** You don't need to be a designer, but a coherent type scale, deliberate color choices, decent spacing, and a layout that holds up at mobile width. Plain CSS, CSS modules, Tailwind, a component library - your call, no requirement either way. The thing being graded is the result, not the tool.

README requirements

1. **How to run** - install command, dev command, the `VITE_API_BASE` env var, any backend setup (CORS, etc).
2. **Tier targeted** - bronze, silver, or gold.
3. **Design decisions** - at least 4 bullets. Good topics: routing approach (silver+), optimistic-update strategy (silver+), real-time approach (gold), how you handled the `X-Username` "not real auth" reality from the user's perspective.
4. **Where my agent helped most and where I had to push back** - one paragraph. Not for grading - I want to read these as a class. Specifics like "Claude wanted to put all the fetch calls inline in components and I had to argue it into a hooks layer" are great.
5. **Silver/gold only:** test command, and (gold) a sentence on each of the gold items you picked.

What to submit

1. Create a branch: `git checkout -b bbs-frontend-<your-github-username>`
2. Put your code in: `assignments/bbs-frontend/<your-github-username>/`
3. Commit your `node_modules` ... just kidding. `.gitignore` them. Vite's scaffolder ships a reasonable `.gitignore`; trust it.
4. Push and open a PR titled "BBS Frontend - <your-name>"

A note on coding agents

The thing about a frontend is that an agent can produce something that looks done in 20 minutes. It mounts, it renders, it talks to the API, the happy path works. But then you tab away, the cache goes stale, you click delete twice fast, you submit an empty form, the network blips - and the seams show. Most of bronze is really there to make you actually click through those edge cases yourself, because your agent will not. It will write the code that handles them only if you ask, specifically, for each one. Push your agent on the loading/error states. Push it on what happens when you click the same button twice. Push it on what the page looks like at 320px wide. The README question about "where I had to push back" is real - I am genuinely curious which corners your particular agent cuts.

The other half of this is that I'll be asking you about your code. Agents are great at producing working code and bad at producing an explanation of why it's structured the way it is, because the structure often emerged from whatever it generated first. So step back before submitting and make sure you can actually narrate what your app is doing - which fetch lives where, what gets re-rendered when the user posts a message, what your app does if the backend goes away for 30 seconds. If you can't answer those, that's information - go simplify until you can.

Grading

Bronze	Satisfactory: a working, usable app with the basics handled - loading states, errors surfaced, every endpoint wired up, validation and identity persistence working
Silver	Good: real architectural choices made and defended, with tests, and the basics of accessibility
Gold	Excellent: pick at least two of - real-time updates, an invented UI feature, automated end-to-end tests, visual design with a point of view

Point breakdown is the same shape as A2: 25 bronze, 10 style/quality, 8 silver, 7 gold, total 50. Style/quality on this one weighs visual taste and code organization more than A2 did - if your app looks like an unstyled form pile or your `App.tsx` is 800 lines, that lands here.